1. Introduction

UAVs, unmanned aerial vehicles, also known as drones, are widely used to capture the high quality photographs required to generate 3D models of buildings, including structures at sites of cultural significance. Before the UAV flies, a planning algorithm produces a set of camera viewpoints intended to capture the whole surface from multiple angles. The term camera viewpoint, or just viewpoint, means the UAV camera position together with its orientation, where it is pointing. This fully specifies what the UAV will see at that point.

Whether such a plan is any good cannot be judged from a list of coordinates alone. The UAV operator needs to see whether every part of the surface is observed often enough, from enough different directions, at a sensible distance and viewing angle.

In this project you will build an interactive tool for inspecting and diagnosing such a plan before deployment, so that a user can find the weak parts of a plan and test whether an extra camera fixes them.

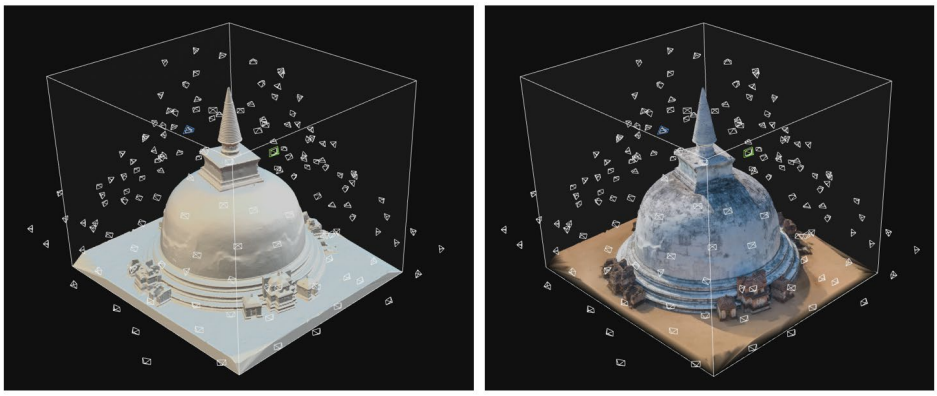


Figure 1. The survey structure without texture and with texture, and the planned viewpoints drawn as oriented camera frustums.

You will need to include all implementation steps, design rationale, challenges encountered, and solutions developed to be documented comprehensively in your project report. Appropriate usage instructions should also be included in your project GitLab README.

Every technical term used below, such as pose, picking, sector, weight or visibility, is defined in plain language in the glossary document GLOSSARY.docx inside the Resources folder. Read it once before this brief.

Provided, and not marked. The Resources folder contains two things. The first is the scaffold library viewscope_core, which supplies every formula, constant and geometric computation in this project. That covers pose construction and projection, offscreen photographic and depth rendering, the camera visibility test with occlusion, sector assignment, weight and score computation, the safe placement test and validating loaders for every file format. You import the library and call it. You never reimplement, modify or tune any of its mathematics, and every model constant lives inside it. The second is the data package for the Kiri Vehera scene. It holds the coloured survey mesh,

the 60000 surface targets, the plan of 185 viewpoints, the collision free flight volume, and all numbers precomputed for that plan, being the visibility of every target from every viewpoint, and for every observing camera its weight with both factors, its sector under eight and under four bins, the best camera per sector, and the per-target diversity and quality score.

Yours, and all of it marked. The architecture, the whole user interface, picking and selection, drawing the plan, surface colouring and heatmaps, the sector and weight visualisations, the first person view, photo export, manual camera placement, deciding when to call the library and caching what it returns, incremental update, threading and process orchestration, packaging, tests and documentation. No scaffold application is supplied, so standing up the GUI, 3D and packaging toolchain is part of the project.

In one sentence, the library computes, and your application decides when to compute, remembers what was computed, and shows it.

2. The data you are given

The surface is represented by surface targets. These are 3D points spread evenly over the scanned surface, each with an outward normal, the direction sticking straight out of the surface at that point. Coverage is judged at these points. For every target the data package lists the viewpoints that can see it, with occlusion by the building already taken into account. Each observing camera arrives with precomputed values that you display rather than compute.

- **A sector**, one of a fixed number of equal directional wedges measured around the target normal, like compass directions around the point. Eight is the default and four is also supplied. Cameras in different sectors see the target from genuinely different directions, which is what photogrammetry needs.

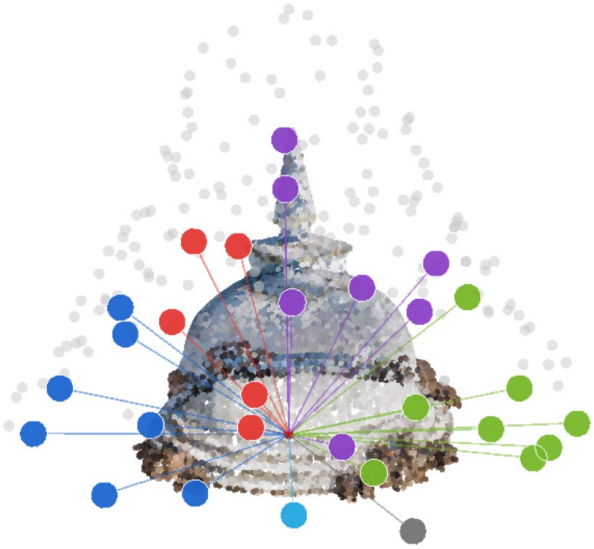


Figure 2. Sector decomposition. The cameras observing one surface target, coloured by their directional sector relative to the target normal.

- **A weight** between zero and one, saying how useful that camera is for that target. It is supplied together with its two factors, one for distance and one for viewing angle, so your interface can show which of the two is holding a camera back.

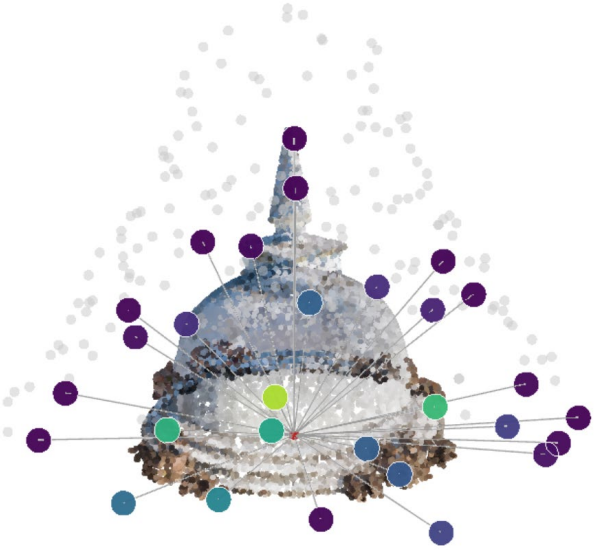


Figure 3. Weight colouring. The viewpoints observing one surface target, coloured by their weight for that target, brighter meaning higher.

For any camera the user adds or moves there are no precomputed values. Your application must obtain them at interaction time by calling the library, which computes visibility, weights, sectors and scores for you. The division of labour is always the same. Shipped viewpoints are looked up. Edited viewpoints are recomputed through the library. The data package also carries the collision free flight volume, a voxel grid marking where a UAV may safely be: outside the building, clear of it by the safe distance, above the minimum altitude, and reachable from the outside airspace. Every planned viewpoint lies inside this volume.



Figure 4. The collision free flight volume enclosing the structure, shown with the planned viewpoints.

3. Core Requirements

The basic suggested approach, which you are free to modify, although you should be able to explain your approach clearly, and you must credit any external sources used, is as follows:

1) Design a system architecture organising the 3D rendering, user interaction and data-processing components, with a class hierarchy whose modules can change independently behind clear interfaces. Document your decisions, in particular how expensive work is kept off the interface and how one camera moving propagates to every dependent view.

2) Load and render the 3D scene. The scene mesh and the surface targets share one frame and scale, so do not recentre or rescale either. The coloured triangle mesh is for display and photography. The surface targets are used for coverage colouring. Include a ground reference plane, coordinate axes, and orbit, pan and zoom.

3) Load and visualise the viewpoint plan. Draw every viewpoint as a small oriented frustum reflecting its yaw and pitch, because a dot without orientation is not acceptable. Provide a viewpoint list showing identifier, position and orientation, selection synchronised in both directions with the 3D scene.

4) Load and validate the data package through the library loaders, failing closed with an actionable message on any malformed file rather than continuing with partial data.

5) Pick a surface target. Picking means clicking in the 3D view and working out what was clicked. Clicking the surface selects the nearest target, marks it, and reveals every camera that observes it, in the 3D scene and in a panel listing the camera identifiers with their values.

6) Visualise the sector decomposition of ONE selected target, as in Figure 2. One point per observer, placed by direction, radius carrying distance, a reference ring at the preferred distance, coloured by sector, with the angular diversity figure shown. The sector count must be switchable between four and eight.

7) Visualise the weights. Colour the observers by weight in the 3D scene, as in Figure 3, and mark the best camera in each sector. Add a bar chart of the best weight per sector with the average shown, so empty sectors read as gaps, and a panel breaking a selected camera into its distance and viewing-angle factors. All of these views must be linked in both directions with the 3D scene and the list.

8) Display live visibility into your tool. The library function that decides which surface targets a camera can see is provided, occlusion included. Your job is the engineering around it. Call it correctly for every camera the user adds or moves, keep its input geometry and camera model exactly as shipped, cache the result, and prove your wiring is right by recomputing one shipped viewpoint and matching the packaged visibility for it exactly. The heavy call must never run on the interface thread.

9) Highlight the visible surface targets of the selected camera in the 3D scene, so the user can see at a glance what that camera covers. Occluded targets must never be highlighted.

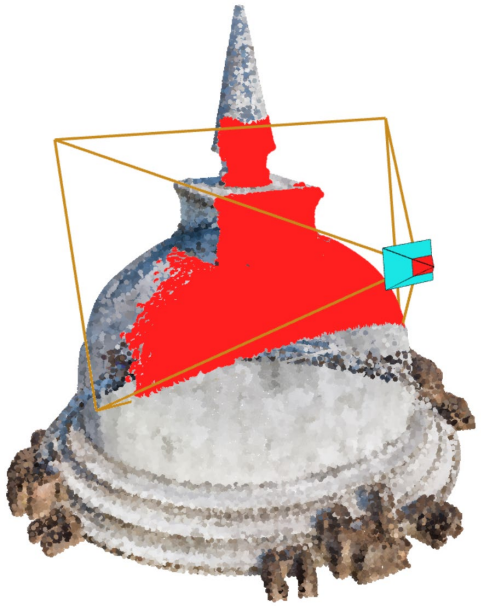


Figure 5. Camera visibility. The surface targets visible to one selected camera highlighted in the 3D scene. Occluded targets are never highlighted.

10) Provide a first person view rendering the coloured model from the default selected viewpoint through the library renderer, in a panel separate from the main viewport.

11) Export photographs in batch, one image per viewpoint at a chosen resolution and directory, with live progress, working cancellation and a usable interface throughout. The first person view and the exporter must share one rendering path.

12) Provide a heatmap mode colouring the surface targets by target quality score, with a legend, an adjustable threshold for weak observation, and a way to navigate to the collection of surface targets with weak observation rather than simply counting them.

13) Support manual viewpoint placement, by picking in the scene or entering a pose, and repositioning. For a new camera, obtain its visibility, weights, sector assignments, highest weight and scores through the library, then update every affected view.

14) Build the user interface with at least these panels, dockable and resizable: the 3D viewport, which is the window the scene is drawn in, the viewpoint list, a target detail panel with the weight factor breakdown, the sector plot, the sector bar chart, a metrics panel, and a parameter panel carrying the weak-observation threshold, the safe distance and the sector count with sensible defaults already set.

15) Keep the interface responsive, graded against fixed thresholds. Navigation must not drop frames, every click must give visible feedback within 0.2 seconds, and anything over 1 second must show progress and offer cancellation while the rest of the interface stays usable. This needs threads, and separate processes where the library renderer is

involved, with clean shutdown. Caching, incremental update and deferred recomputation are yours to design.

16) VTK framework – it is advised to use VTK (Visualization Toolkit) for UI/UX since this is very fast and will avoid performance issues.

4. Mastery Work

The remaining 20% of the marks are for work beyond this, and the amount awarded depends on the complexity of the added features. These are engineering extensions rather than new mathematics. For example:

- **Curvature-aware flight path:** Order the viewpoints into a sequence and generate a smooth path with constant-speed traversal, which requires arc-length reparameterisation, and show heading indicators.
- **Live coverage accumulation:** Fly the sequence and update the heatmap step by step as each viewpoint is passed, which needs incremental recolouring rather than a full recompute per step.
- **In-flight photography and video recording:** Show the live first person image while flying, recorded at a chosen frame rate and resolution with an information overlay, without freezing the interface.
- **Interactive viewpoint editing:** Keyboard adjustment of position and orientation, adding and removing viewpoints, real-time update of everything affected, and undo and redo.
- **Other, which you must motivate:** For example sustained interactivity on a much denser set of surface targets with measured evidence, saveable workspace layouts, or a diagnostic report export.

These are independent of one another, and the animation and photography items may follow the raw viewpoint order.

5. Code Structure

- Modularise your code as a pipeline: loading and validation, scene construction, interaction and selection, scoring lookup and aggregation, presentation.
- Keep components replaceable, with the rendering back end, the scoring source and the persistence format each behind an interface. The first person view and the batch exporter must be two callers of one rendering service.
- Use appropriate data structures, meaning NumPy arrays for numerical data and dictionaries for metadata. Consider implementing a plugin architecture for extensibility.
- Treat the library as a dependency. Do not modify it, and report anything you believe is wrong instead of forking it.

6. Resources

The Resources folder contains the scaffold library viewscope_core, the glossary, and the Kiri Vehera data package, being the coloured survey mesh, the 60000 surface targets with normals, the plan of 185 viewpoints, the collision free flight volume, and the full precomputed scoring and visibility for that plan. Your submission is expected to stay interactive on this dataset. All constants of the scoring model live inside the library, so you never enter a numeric model parameter yourself. Angles are in degrees, lengths in metres, and every geometry shares one world frame with nothing recentred or rescaled. The library is installed with pip install -e as described in the Resources README.

7. Glossary

Triangle mesh. A 3D surface stored as points (vertices) joined into triangles (faces). data/kiri/scene.obj is a triangle mesh in which every vertex also carries a colour, so the model displays with its real scanned appearance.

Surface targets. 60000 individual 3D points spread evenly over the scanned surface, each with an outward normal (see below). They are the measurement grid: coverage is judged at these points, never at mesh vertices. Stored in data/kiri/targets.ply. They lie ON the same surface the mesh represents and share its coordinate frame exactly.

Normal. The direction sticking straight out of the surface at a point, stored as a unit vector. A camera can only usefully observe a target from the side its normal points towards.

Viewpoint (camera viewpoint). One planned UAV camera shot: a position in metres plus an yaw and a pitch. That fully specifies what the camera sees. The plan in data/kiri/plan.json is 185 of these.

Yaw and pitch. Yaw is the compass-like rotation about the vertical axis (measured from the +X axis towards +Y). Pitch tilts the view up (positive) or down (negative). Roll is always zero in this project.

Pose. Position plus orientation packaged as one 4 x 4 matrix, produced by viewscope_core.camera_pose(position, yaw_deg, pitch_deg). You never build this matrix yourself. You ask the library for it.

Frustum. The pyramid-shaped volume a camera can see, with its tip at the camera. Drawing a viewpoint "as a frustum" means drawing that little pyramid so its orientation is visible at a glance.

Rendering. Turning 3D geometry into a 2D image. The library does every photographic render for you (SceneRenderer). What you build in VTK is the interactive viewport, which VTK itself renders.

Offscreen rendering. Rendering into memory instead of a window. The library uses it to produce photographs and depth maps without any GUI.

Depth map. An image where each pixel stores the distance to the nearest surface instead of a colour. It is how occlusion is decided: anything farther than the recorded depth at its pixel is hidden.

Visibility. Whether a camera actually sees a target: the target must be in front of the camera, inside the image, on the side its normal faces, and not hidden behind the building (checked against the depth map).

Occlusion. Being hidden behind geometry. An occluded target is not visible even if the camera points straight at it.

Picking. Clicking in the 3D view and working out which object was clicked. "Pick a surface target" means: the user clicks on the model and your tool selects the nearest surface target to the clicked point.

Sector (directional bin). The compass directions around a target's normal, divided into equal wedges: eight by default, four as the coarse alternative. Every observing camera falls in exactly one wedge (sector_indices). Cameras in different sectors see the target from genuinely different directions, which is what multi-view reconstruction needs.

Weight. A number between 0 and 1 saying how useful one camera is for one target (weight_factors).

Target score. The per-target quality figure: the mean over all sectors of the best weight in that sector, empty sectors counting zero (target_score). This is what the heatmap colours.

Safe placement. The rule refusing a new camera inside the building or closer than the safe distance to the surface (check_placement).

World frame. The single right-handed coordinate system every file shares: +Z up, metres, angles in degrees. Nothing is ever recentred or rescaled, so geometry from any two files can be drawn together directly.

Heatmap. Colouring the surface by target score so weak regions stand out as a colour band rather than a number.